CE 222 – *Computer Organization and Assembly Language (COAL)*
Lecture 13

Dr. Asad Mahmood

Feb. 16, 2026

Lecture Outline

- Announcements (Assignment and Quiz 2 Next week – Chapter 2)
- Outline of Previous Lecture:
  - **Chapter 2 - Instructions: Language of the Computer**
    - 2.8 Supporting Procedures in Computer Hardware

- Outline to Today's Lecture:
  - **Chapter 2 - Instructions: Language of the Computer**
    - 2.8 Supporting Procedures in Computer Hardware

Chapter 2

Instructions: Language of the Computer

Chapter Outline

- **2 Instructions: Language of the Computer**
  - 2.1 Introduction
  - 2.2 Operations of the Computer Hardware
  - 2.3 Operands of the Computer Hardware
  - 2.4 Signed and Unsigned Numbers
  - 2.5 Representing Instructions in the Computer
  - 2.6 Logical Operations
  - 2.7 Instructions for Making Decisions
  - 2.8 Supporting Procedures in Computer Hardware
  - 2.9 Communicating with People
  - 2.10 RISC-V Addressing for Wide Immediates and Addresses
  - 2.11 Parallelism and Instructions: Synchronization
  - 2.12 Translating and Starting a Program
  - 2.13 A C Sort Example to Put it All Together
  - 2.14 to 2.24 …..

Procedures

- Procedures in assembly language is one tool programmers use to structure programs for
  - Ease of understanding
  - Code re-reuse
- Similar to *functions/methods* in high level languages like C, Python
- Parameters act as an interface between the procedure and the rest of the program

- Spy analogy – cover the tracks!

Procedure Calling

- Following steps are followed when calling procedures:
  1. Place parameters in registers (x10 to x17)
  2. Transfer control to procedure / callee
  3. Acquire storage for procedure
  4. Perform procedure's operations
  5. Place result in register (x10 to x17) for caller
  6. Return to place of call (address in x1)

Procedure Call Instructions

- Some dedicated RISC-V instructions for procedures
- To call a procedure: jump and link (`jal`)

  `jal x1, ProcedureLabel`
  - Address of following instruction put in x1
  - Jumps to target address
- To return from a Procedure: jump and link register (`jalr`)

  `jalr x0, 0(x1)`
  - Return address?
- Program counter (PC): A register to store the address of current instruction being executed
  - Value in x1 wrt PC

Using more Registers

- If a compiler needs more registers for a procedure than the eight argument registers, portion of memory/RAM used – termed spilling to the memory!
- A special portion of memory, known as Stack, is used for such purpose:
  - Stack works as a last-in-first-out queue.
  - Stack Pointer (register x2)
  - Push – place data on stack
  - Pop – Remove data from stack
  - Grows downwards!

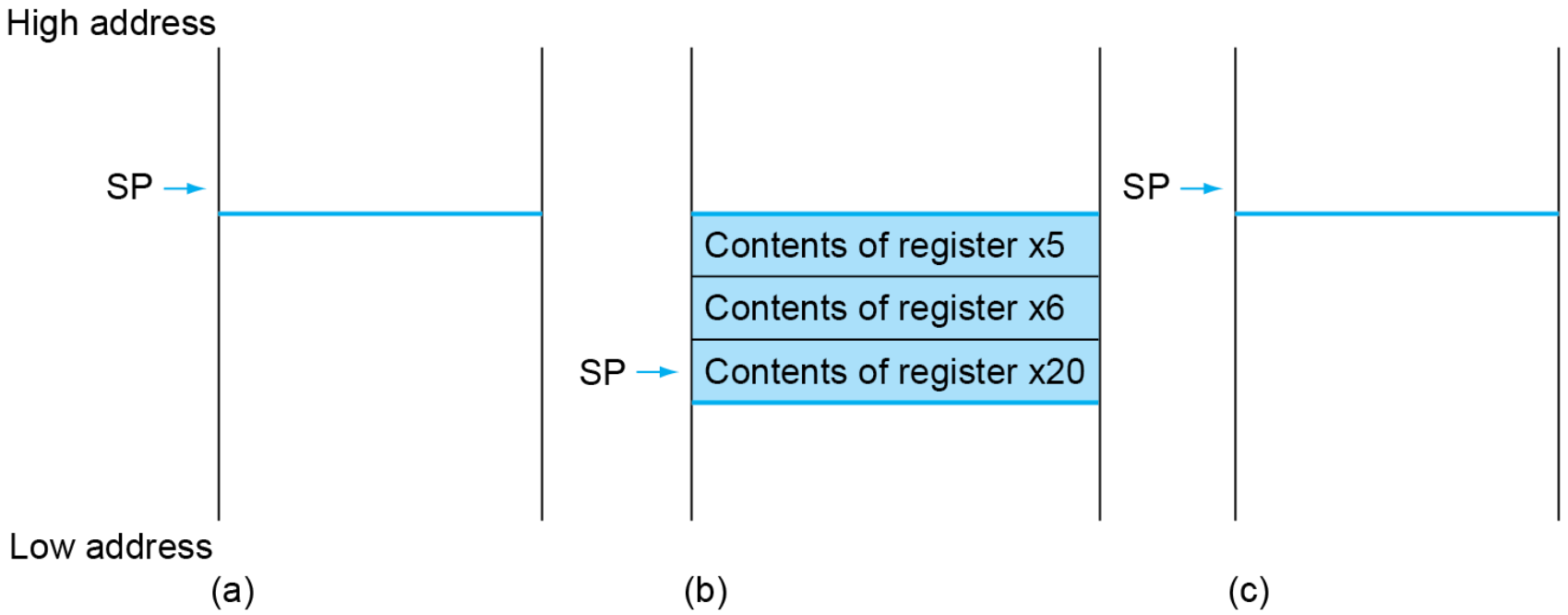
Leaf Procedure Example

- C code:

```
int leaf_example (int g, int h, int i, int j)
{
  int f;
  f = (g + h) - (i + j);
  return f;
}
```

  - Arguments g, …, j in x10, …, x13
  - f in x20
  - temporaries x5, x6
  - Need to save x5, x6, x20 on stack
- RISC-V Code

Local Data on the Stack



Register Usage

- x5 – x7, x28 – x31: temporary registers
  - Not preserved by the callee
- x8 – x9, x18 – x27: saved registers
  - If used, the callee saves and restores them

Non-Leaf Procedures

- Procedures that call other procedures
- For nested/recursive calls, a conflict in parameters and return address registers can occur -> avoided via use of stack!
- For nested call, caller needs to save on the stack:
  - Its return address
  - Any arguments and temporaries needed after the call
  - Stack pointer (sp) updated accordingly!
- Restore from the stack after the call

Non-Leaf Procedure Example

- C code:

```
int fact (int n)
{
  if (n < 1)
        return 1;
  else
        return n * fact(n - 1);
}
```

  - Argument n in x10
  - Result in x10

Non-Leaf Procedure Example

- RISC-V code:

```
fact:
    1 addi sp,sp,-8
    2 sw   x1,4(sp)
    3 sw   x10,0(sp)
    4 addi x5,x10,-1
    5 bge  x5,x0,L1
    6 addi x10,x0,1
    7 addi sp,sp,8
    8 jalr x0,0(x1)
9 L1: addi x10,x10,-1
   10 jal  x1,fact
   11 addi x6,x10,0
   12 lw   x10,0(sp)
   13 lw   x1,4(sp)
   14 addi sp,sp,8
   15 mul  x10,x10,x6
   16 jalr x0,0(x1)
```